



Adaptive Bilateral Texture Filter

for Image Smoothing

Implementation and Evaluation Based on Xu et al. (2022)

Egypt-Japan University of Science and Technology

Computer Vision Project

Outline

- 1 Introduction
- 2 Pipeline Overview
- 3 Gradient Map
- 4 Structure Map
- 5 Adaptive Scale Map
- 6 Fourier Approximation
- 7 Filtering
- 8 Implementation
- 9 Results
- 10 Conclusion

Problem Statement

Traditional image smoothing filters such as:

- Gaussian Blur
- Mean (Average) Filter
- Standard Bilateral Filter

often produce undesirable effects:

- Remove textures and edges together
- Blur important image structures
- Reduce image sharpness

Goal of this work:

Main Objective

Smooth texture regions while preserving important structural edges.

Paper Title:

Adaptive Bilateral Texture Filter for Image Smoothing

Authors:

Xu, H., Zhang, Z., Gao, Y., Liu, H., Xie, F., and Li, J.

Published In:

Frontiers in Neurorobotics, Volume 16, Article 729924 (2022)

Main Contribution:

- Adaptive smoothing based on local image structure
- Fourier acceleration for efficient filtering
- Near constant-time complexity

Complete Pipeline

The implementation follows the pipeline proposed in the paper:

- 1 Load and preprocess image
- 2 Compute gradient map
- 3 Compute structure map
- 4 Generate adaptive scale map
- 5 Approximate Gaussian kernel using Fourier series
- 6 Apply adaptive bilateral filtering
- 7 Evaluate filtering results

Each stage was visualized and evaluated independently.

Stage 1 – Gradient Map

The image is first smoothed using a Gaussian filter.
Then the gradient magnitude is computed:

$$G_p = \sqrt{(\partial_x f_\sigma)^2 + (\partial_y f_\sigma)^2}$$

Purpose of this stage:

- Detect intensity variations
- Highlight edges and texture details
- Provide structural information for later stages

Gradient Map Interpretation

High Gradient Values

Strong edges and structural boundaries

Low Gradient Values

Flat or smooth image regions

The gradient map alone cannot distinguish between:

- Important structural edges
- Fine textures

Therefore, an additional structure analysis step is required.

Stage 2 – Structure Map

Four directional neighborhoods are examined:

- North-West (NW)
- North-East (NE)
- South-West (SW)
- South-East (SE)

For each direction:

$$A_p^{(j)} = \frac{\sum g_m(p, b) \cdot G_b}{\sum g_m(p, b)}$$

Final structure measure:

$$S_p = \max_j A_p^{(j)}$$

Purpose of Structure Map

The structure map distinguishes between:

Texture Regions

Small repetitive variations

Structural Edges

Important object boundaries

Interpretation:

- Large $S_p \rightarrow$ strong structural edge
- Small $S_p \rightarrow$ texture region

Stage 3 – Adaptive Scale Map

Adaptive window size:

$$W_p = \max \left(\eta \left(\frac{1}{\lambda} \right)^{S_p^2}, \delta \right)$$

Where:

- W_p : spatial kernel radius
- η : maximum radius
- λ : decay factor
- δ : minimum radius

Adaptive Filtering Behavior

Texture Regions

Large smoothing window

$$W_p \uparrow$$

Aggressive smoothing

Edge Regions

Small smoothing window

$$W_p \downarrow$$

Edge preservation

Fourier Approximation

The Gaussian range kernel is approximated using a Fourier series:

$$\hat{\phi}(t) = \sum_{n=-N}^N c_n e^{inv t}$$

Advantages:

- Reduces computational cost
- Enables fast implementation
- Makes integral-image acceleration possible

Adaptive Bilateral Filtering

The final filtered image is computed as:

$$\hat{u}(p) = \frac{E(p)}{H(p)}$$

Where:

$$E(p) = \sum c_n e^{-invf(p)} e_n(p)$$

$$H(p) = \sum c_n e^{-invf(p)} h_n(p)$$

Integral Image Acceleration

Integral images are used to compute local sums efficiently.

Advantages:

- Constant-time window summation
- Significant runtime reduction
- Efficient adaptive filtering

This allows the algorithm to achieve near:

$$O(1)$$

complexity with respect to window size.

Implementation Details

The project was implemented using Python in Google Colab.

Libraries used:

- NumPy
- OpenCV
- SciPy
- scikit-image
- Matplotlib

Features implemented:

- Full pipeline visualization
- Runtime measurements
- Multi-iteration filtering
- Automatic image export

Filtering Results

Observed results:

- Texture regions successfully smoothed
- Important edges preserved
- Reduced clutter in edge detection
- Enhanced structural clarity

Additional applications:

- Detail enhancement
- Edge detection preprocessing

Timing Analysis

Runtime was measured for different window sizes:

$$\eta = 8, 12, 16$$

Goal:

- Verify near constant-time complexity
- Compare runtime scalability

Observation:

Result

Filtering time remained nearly constant despite increasing window size.

Conclusion

- Successfully implemented the Adaptive Bilateral Texture Filter
- Reproduced the major stages of the paper
- Preserved structural edges while smoothing textures
- Verified efficient runtime behavior

Future improvements:

- GPU acceleration
- Parallel implementation
- Real-time processing optimization



Xu, H., Zhang, Z., Gao, Y., Liu, H., Xie, F., and Li, J. (2022).
Adaptive bilateral texture filter for image smoothing.
Frontiers in Neurorobotics, 16, 729924.



Eastman Kodak Company. (1999).
Kodak Lossless True Color Image Suite.
Available at: <https://r0k.us/graphics/kodak/>

Thank You

Questions?

